The Python Ecosystem (Part 2)

Alexis Shakas Ian Wasser Hagen Söding

ETH Zurich

2026-04-28

Lecture	Date	T.H.A.	Room
1: Kickoff	17/2	N	RZ D8
2: AI assistance and tools	24/2	N	RZ D8
3: Project-based teamwork	3/3	N	RZ D8
4: Intro to Git	10/3	N	RZ D8
5: Git continued	17/3	N	ML H41.1
6: Git collaboration	24/3	N	RZ D8
7: AI-assisted Python	31/3	Y	ML H41.1
8: The 3 R's in Python	14/4	Y	ML H41.1
9: The Python ecosystem	21/4	Y	RZ D8
10: Python ecosystem (Part 2)	28/4	**N**	RZ D8
11: Testing and debugging	5/5	**Y**	ML H41.1
12: OOP	12/5	Y	RZ D8
13: Project distribution	19/5	N	RZ D8
14: Project marketplace	26/5	N	RZ D8

T.H.A.: Take home assignment and peer review **today**.

A new teaching assistant: Hagen Söding

- 2017: B.Sc. Applied Mathematics
- 2019: M.Sc. in Mathematics
- 2026: Ph.D. in Geophysics
- Since: Posdoc in Geophysics

- Applied mathematician; now also geophysicist.
- PhD work: subsurface mapping strategies with focus on detecting and tracking sequestered CO_2.
- Interested in (elegant) math for environmental problems.

I've enjoyed teaching ever since my own studies, and I look forward to helping you today (and perhaps in the future as well)!

Where we left off

Last week we covered **the structure of a Python package**:

- The standard project layout (`my_project/`, `data/`, `tests/`, `docs/` ...)
- What goes in `pyproject.toml` and `requirements.txt`
- Why we need virtual environments (the **apartment** analogy)
- A shallow look at the `__init__.py` file

Plan for today

1. **Namespace packages** — what happens **without** an `__init__.py`
2. **A closer look at the `__init__.py` file**
3. **Paths**
4. **Imports — the deeper picture**
5. **Best practices** for evaluating package quality before you `pip install`

In the second half of the lecture, you will work on your project while each team meets with me in a breakout room for team-on-1 feedback (Breakout rooms announced at the end of the first part of the lecture.)

Namespace packages

A folder without `__init__.py`

What if we make a Python package, but **forget** the `__init__.py` file?

A folder without `__init__.py`

What if we make a Python package, but **forget** the `__init__.py` file?

Folder structure:

```
experiment/
└── my_project/
    └── greetings.py
```

```python
# my_project/greetings.py
def hello(name: str = "Bob"):
    return f"Hello, {name}!"
```

No `__init__.py` anywhere.

What happens when we try to `import my_project.greetings` from inside `experiment/`?

MiniTask: Try a folder without __init__.py

1. Create a folder somewhere (e.g. `experiment/`).
2. Inside it, create a sub-folder `my_project/` with no `__init__.py` but containing one file `greetings.py`:

```
def hello(name: str = "Bob") -> str:
    return f"Hello, {name}!"
```

3. From inside `experiment/`, start Python and run:

```
from my_project.greetings import hello
print(hello())
```

What just happened?

The folder `my_project/` had no `__init__.py`, but Python still treated it as a package. This is called a **namespace package** (PEP 420).

What just happened?

The folder `my_project/` had no `__init__.py`, but Python still treated it as a package. This is called a **namespace package** (PEP 420).

Why does it exist?

To let **one umbrella name** be split across many pip-installable pieces. Examples: `azure.*` or `google.cloud.*` — each subpackage is a separate install, but they all share the same top-level name.

Why not for your projects?

- Implicit and easy to forget
- Slower imports (Python keeps scanning `sys.path`)
- No place to define constants or run setup code

The `__init__.py` file

Making it a regular package

Drop an `__init__.py` (even empty) into the folder, and it becomes a **regular package**.

```
experiment/
└── my_project/
    ├── __init__.py   ← explicit
    └── greetings.py
```

The same import still works:

```
from my_project.greetings import
hello
print(hello())
```

But now there's a **file** you control, where you can place setup code, expose constants, or re-export functions.

When does it run?

__init__.py runs **the first time** the package is imported.

```
import my_project                # __init__.py runs once
from my_project import greetings  # already loaded
from my_project.greetings import hello
```

When does it run?

`__init__.py` runs **the first time** the package is imported.

```
import my_project                # __init__.py runs once
from my_project import greetings  # already loaded
from my_project.greetings import hello
```

This makes it the natural place to put **anything that should be set up once** when the package is loaded — for example, **project-wide paths**.

Paths

Why paths matter

You already met this in **Assignment 3 — Exercise 2**: the monolith script had hardcoded Windows paths and broke as soon as it left the coder's laptop.

```
# Works on Bob's laptop
open("C:\\Users\\Bob\\data\
\input.csv")

# Breaks everywhere else.
```

Hardcoded paths are not reproducible across:

- different separators (\ vs /)
- different users
- different machines

pathlib

Python's **standard library** ships with `pathlib`. It treats paths as objects and handles OS differences for you.

```
from pathlib import Path

p = Path("data")

# The slash operator joins paths — works on any OS
filepath = p / "my_data.csv"
nested   = p / "audio" / "song.mp3"
```

Other useful operations:

```
filepath.exists()              # does this file exist?
filepath.parent                # the directory containing it
filepath.suffix                # ".csv"
filepath.with_suffix(".json")  # swap extension
```

`Path(__file__)`

Every Python file has a special variable: `__file__` — the **path of the file itself**.

```python
# inside my_project/__init__.py
from pathlib import Path

HERE = Path(__file__).resolve()
print(HERE)
# /home/alexis/my_project/my_project/__init__.py
```

`Path(__file__)`

Every Python file has a special variable: `__file__` — the **path of the file itself**.

```
# inside my_project/__init__.py
from pathlib import Path

HERE = Path(__file__).resolve()
print(HERE)
# /home/alexis/my_project/my_project/__init__.py
```

`Path(__file__)` always points to **the file it is written in**, no matter which folder Python was started from. We can use this to anchor **project paths**.

Example code that can go in `__init__.py`

```python
# my_project/__init__.py
from pathlib import Path

# Path of the package directory (where this file lives)
PACKAGE_DIR = Path(__file__).resolve().parent

# Path of the project directory (one level up)
PROJECT_DIR = PACKAGE_DIR.parent

# Specific directories
DATA_DIR   = PROJECT_DIR / "data"
OUTPUT_DIR = PROJECT_DIR / "output"

# Make sure they exist
DATA_DIR.mkdir(parents=True, exist_ok=True)
OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
```

MiniTask: Add project paths to the same __init__.py

Continue with the `experiment/my_project/` project from the first minitask.

1. In the same `my_project/__init__.py`, paste the snippet from the previous slide.
2. From inside `experiment/`, start Python and run:

```
from my_project import DATA_DIR, OUTPUT_DIR
print(DATA_DIR)
print(OUTPUT_DIR)
```

3. Check your folder — there should now be `data/` and `output/` directories next to `my_project/`. Who created them?
4. Delete one folder and re-run the import. What happens?

Is this good practice?

Yes, for project code. It's pragmatic, common, and exactly what you need:

- One **single source of truth** for project paths
- Anywhere in your code: `from my_project import DATA_DIR`
- Works on any machine, any OS, any current directory

With one caveat. `__init__.py` runs **every time** the package is imported, including the `mkdir` calls.

For **library** code shipped to others, prefer keeping `__init__.py` minimal. For **your project**, where you control how it's used, this pattern is fine.

Imports — the deeper picture

How Python finds a module

When you write `import my_project`, Python searches in order:

How Python finds a module

When you write `import my_project`, Python searches in order:

1. `sys.modules` — already-imported modules (a cache)

How Python finds a module

When you write `import my_project`, Python searches in order:

1. `sys.modules` — already-imported modules (a cache)
2. Built-in modules (`sys`, `math`, …)

How Python finds a module

When you write `import my_project`, Python searches in order:

1. `sys.modules` — already-imported modules (a cache)
2. Built-in modules (`sys`, `math`, …)
3. Each directory in `sys.path`, in order

How Python finds a module

When you write `import my_project`, Python searches in order:

1. `sys.modules` — already-imported modules (a cache)
2. Built-in modules (`sys`, `math`, …)
3. Each directory in `sys.path`, in order

```
import sys
print(sys.path)
# ['', '/path/to/your/script/dir',
#  ...PYTHONPATH entries...,
#  ...site-packages (where pip installs)]
```

How Python finds a module

When you write `import my_project`, Python searches in order:

1. `sys.modules` — already-imported modules (a cache)
2. Built-in modules (`sys`, `math`, …)
3. Each directory in `sys.path`, in order

```
import sys
print(sys.path)
# ['', '/path/to/your/script/dir',
#  ...PYTHONPATH entries...,
#  ...site-packages (where pip installs)]
```

If nothing matches ⟶ `ModuleNotFoundError`.

Absolute vs relative imports

Inside **your own package** (`my_project`), you can write the same import two ways:

Absolute — full path from the project root:

```
# inside my_project/analysis.py
from my_project.greetings import
hello
from my_project import DATA_DIR
```

Clear, but verbose.

PEP 8 **prefers absolute imports**!

Relative — using dots:

```
# inside my_project/analysis.py
from .greetings import hello
from . import DATA_DIR
```

`.` = current package, `..` = parent.

The shadowing trap

Naming a file the same as a stdlib module breaks everything.

The shadowing trap

Naming a file the same as a stdlib module breaks everything.

You have a file `math.py` in your project:

```
my_project/
├── math.py
└── analysis.py
```

Then this fails:

```python
# analysis.py
import math
print(math.pi)
# AttributeError: module 'math' has
# no attribute 'pi'
```

Python found **your** `math.py` first.

Avoid filenames that match stdlib modules: `math.py`, `string.py`, `random.py`, `email.py`, `json.py`, …

PEP 8: organising your imports

Group your imports in three blocks, separated by a blank line:

```
# 1. Standard library
import os
from pathlib import Path

# 2. Third-party packages (pip-installed)
import numpy as np
import requests

# 3. Local project code
from my_project import DATA_DIR
from my_project.greetings import hello
```

Inside each group, most Python projects follow alphabetical order.

Module vs symbol: which to import?

The same function, two import styles. Both are correct — but they read **differently**.

Import the module:

```
from my_project import greetings

greetings.hello()
greetings.goodbye()
```

1. Every call says **where it comes from**.
2. Best when you use **several** things from the module.

Import the symbol:

```
from my_project.greetings import
hello

hello()
```

1. Shorter at the call site.
2. Best when you use **one specific thing** repeatedly.

MiniTask: Install your package and try both import styles

We will **install** the `experiment/my_project` package so it works from any folder.

1. In `experiment/`, download the `pyproject.toml` from moodle.
2. (optional) Create a virtual environment and activate it!
3. From inside `experiment/`, run `pip install -e ..`
4. `cd` to **any other folder** on your computer, start Python and try **both** styles:

```
from my_project import greetings
greetings.hello("L10")

from my_project.greetings import hello
hello("L10")
```

5. Try `from my_project import DATA_DIR; print(DATA_DIR)`.

Installing python packages

Before you `pip install`...

PyPI has **600,000+** packages. Most are abandoned, half-written, or just plain bad. Spend 30 seconds checking before you install.

A quick checklist on the package's PyPI page:

Before you `pip install`...

PyPI has **600,000+** packages. Most are abandoned, half-written, or just plain bad. Spend 30 seconds checking before you install.

A quick checklist on the package's PyPI page:

- **Supported Python versions** — does it work with your version?

Before you `pip install`...

PyPI has **600,000+** packages. Most are abandoned, half-written, or just plain bad. Spend 30 seconds checking before you install.

A quick checklist on the package's PyPI page:

- **Supported Python versions** — does it work with your version?
- **Last release date** — anything older than 2 years is suspect

Before you `pip install`...

PyPI has **600,000+** packages. Most are abandoned, half-written, or just plain bad. Spend 30 seconds checking before you install.

A quick checklist on the package's PyPI page:

- **Supported Python versions** — does it work with your version?
- **Last release date** — anything older than 2 years is suspect
- **License** — present, and compatible with your use?

Before you `pip install`...

PyPI has **600,000+** packages. Most are abandoned, half-written, or just plain bad. Spend 30 seconds checking before you install.

A quick checklist on the package's PyPI page:

- **Supported Python versions** — does it work with your version?
- **Last release date** — anything older than 2 years is suspect
- **License** — present, and compatible with your use?
- **Source link** — leads to a real GitHub repo with activity?

Before you `pip install`...

PyPI has **600,000+** packages. Most are abandoned, half-written, or just plain bad. Spend 30 seconds checking before you install.

A quick checklist on the package's PyPI page:

- **Supported Python versions** — does it work with your version?
- **Last release date** — anything older than 2 years is suspect
- **License** — present, and compatible with your use?
- **Source link** — leads to a real GitHub repo with activity?
- **Description** — has a real README, not just a placeholder?

Wrap-up

What you should be able to do now

After today, you should be able to:

What you should be able to do now

After today, you should be able to:

- Know what a **namespace package** is

What you should be able to do now

After today, you should be able to:

- Know what a **namespace package** is
- Explain what `__init__.py` does and **when** it runs.

What you should be able to do now

After today, you should be able to:

- Know what a **namespace package** is
- Explain what `__init__.py` does and **when** it runs.
- Use `pathlib` to build OS-independent paths.

What you should be able to do now

After today, you should be able to:

- Know what a **namespace package** is
- Explain what `__init__.py` does and **when** it runs.
- Use `pathlib` to build OS-independent paths.
- Write absolute and relative imports, group them by PEP 8, and judge whether a third-party package is worth installing.

This knowledge is important to make your project reproducible and shareable, and to avoid the common pitfalls that break code when it leaves your laptop.

Next week

Lecture 11 — Testing and Debugging.

We go into testing and debugging, and attack a bigger question we'll come back to: **if AI wrote the code AND the tests, what are you actually testing?**

Breakout rooms

After the break, I'll open breakout rooms for each team to meet with me for feedback on your project progress and next steps. Meeting order:

- Team red (14:15:–14:25)
- Team blue (14:25-14:35)
- Team yellow (14:35-14:45)
- Team orange (14:45-14:55)

Note that times may sligthly shift depending on how the class advances to this point.